

Ros2 최종프로젝트 AI 사용 보고서

휴머노이드 수습단원 이주원

목차

1. ai사용 부분
2. 대화내용
3. 깨달은 점

1. ai 사용 부분

이 프로젝트에서 ai가 사용된 부분은 크게 10개다. 첫 번째로 커스텀 메시지를 빌드 설정에 넣는 방법이다. 메시지 파일을 만들었는데 빌드 설정에 어떻게 등록해야 하는지 헛갈려 물어보았다.

두 번째는 서비스 호출이다. 충돌이 일어났을 때 서비스를 부르려고 하는데 콜백 안에서 호출하면 프로그램이 멈춰서 그 이유를 확인했다.

세 번째는 차의 위치를 화면에 옮기는 방법이다. 차선 번호를 받아서 화면의 세로 좌표로 바꿔야 하는데 어떻게 처리할지 물어보았다.

네 번째는 앞차 간격이다. 차선이 여러 개가 되면서 기존 방식이 통하지 않게 되어 어떻게 바꿔야 하는지 확인했다.

다섯 번째는 실행 시 발생한 오류다. 빌드는 성공했는데 실행하면 노드가 죽어서 원인을 물어보았다.

여섯 번째는 플레이어 노드의 구조다. 위치와 충돌을 관리하는 노드를 새로 만들어야 해서 어떤 구성으로 짜야 하는지 확인했다.

일곱 번째는 플레이어가 순간이동하는 문제다. 차에 부딪히면 죽는 모습이 보이지 않고 바로 시작 위치로 튀어서 이유를 물어보았다.

여덟 번째는 타입서포트 라이브러리 오류다. 메시지가 생성되었는데도 라이브러리를 찾을 수 없다는 오류가 나서 확인했다.

아홉 번째는 충돌 판정이다. 두 물체가 겹쳤는지 어떻게 판단하는지 물어보았다.

열 번째는 점수와 난이도 연동이다. 점수가 오를수록 차가 빨라지게 하려면 어떤 구조로 가야 하는지 확인했다

2. 대화 내용

커스텀 메시지 Car, CarArray, Move를 cmake에 어떻게 넣어?

`ament_package()` 위쪽에 이렇게 넣어:

```
cmake
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/Car.msg"
  "msg/CarArray.msg"
  "msg/Move.msg"
  DEPENDENCIES std_msgs
)

rosidl_get_typesupport_target(cpp_typesupport_target ${PROJECT_NAME} "rosidl_typesupport_cpp")

target_link_libraries(${PROJECT_NAME} "${cpp_typesupport_target}")
target_link_libraries(vehicle_node "${cpp_typesupport_target}")
target_link_libraries(player_node "${cpp_typesupport_target}")
```

`player_node` 는 아직 안 만들었으면 그 줄은 빼두고, 만들고 나서 추가해.

순서 주의. `rosidl_generate_interfaces` 는 `add_executable` 들보다 뒤에 와야 해.
`target_link_libraries` 가 이미 존재하는 타겟을 참조하니까.

메시지 파일 목록을 빌드 설정에 적어주고, 그것을 사용하는 실행 파일마다 따로 연결해줘야 한다는 답을 받았다. 순서가 중요해서 실행 파일을 만드는 부분보다 뒤에 와야 한다는 설명도 함께 들었다. 처음에 앞쪽에 적었더니 실행 파일을 찾을 수 없다는 오류가 나서 위치를 옮겼다.

ROS2 노드가 콜백을 실행하는 구조를 보면 이유가 나와.

노드는 `spin()` 안에서 이벤트를 하나씩 꺼내 처리하는 단일 루프로 돌아. 메시지가 오면 구독 콜백을 부르고, 그게 끝나야 다음 이벤트를 꺼내. 서비스 응답이 도착하는 것도 그 "다음 이벤트" 중 하나야.

그래서 구독 콜백 안에서 응답을 기다리면:

```
구독 콜백 실행 중
└─ 서비스 요청 보냄
└─ 응답 올 때까지 대기 ← 여기서 멈춤
    ↑
    응답이 도착해도 처리하려면 스핀 루프로 돌아가야 하는데,
    루프는 지금 이 콜백이 끝나기를 기다리는 중
```

콜백은 응답을 기다리고, 응답은 콜백이 끝나야 처리되는 상태야. 서로를 기다려서 영영 안 풀려. 이걸 데드락이라고 해.

```
cpp
// 이렇게 하면 곤음
auto future = client->async_send_request(req);
rclcpp::spin_until_future_complete(node, future); // 콜백 안에서는 금지
```

`spin_until_future_complete` 는 "이 노드를 스핀시키면서 응답을 기다려라"는 뜻인데, 이미 스핀 중인 노드를 또 스핀시키는 거라 에러가 나거나 그대로 곤어.

해결

기다리지 말고, 응답이 오면 실행할 함수를 같이 넘기면 돼.

```
cpp
client->async_send_request(req,
    [this](rclcpp::Client<MySrv>::SharedFuture future)
    {
        auto res = future.get();
        // 응답 처리
    });
```

요청만 보내고 콜백은 바로 끝나. 스핀 루프가 자유로워지니까, 나중에 응답이 도착하면 루프가 그걸 꺼내서 람다를 실행해줘.

main에서는 왜 되나

```
cpp
```



노드는 이벤트를 하나씩 꺼내 처리하는 단일 루프로 돌기 때문에, 콜백 안에서 응답을 기다리면 응답을 처리해줄 루프 자체가 막혀버린다는 답을 받았다. 콜백은 응답을 기다리고 응답은 콜백이 끝나야 처리되는 상태가 되어 영영 풀리지 않는다고 한다.

해결은 기다리지 말고 응답이 왔을 때 실행할 함수를 같이 넘기는 것이었다. 요청만 보내고 콜백은 바로 끝나기 때문에 루프가 자유로워진다. 예제들이 대부분 기다리는 형태라 그대로 쓰다가 걸리기 쉬운 부분이라는 설명도 들었다.

차의 위치 갱신 쪽 어떻게 해?

Qt 쪽 얘기지? 차선 번호를 y좌표로 바꾸는 게 핵심이야.

1. QNode에서 타입 교체

qnode.hpp

```
cpp
#include "road_friend/msg/car_array.hpp"

rclcpp::Subscription<road_friend::msg::CarArray>::SharedPtr vehicle_sub_;

Q_SIGNALS:
    void vehiclesUpdated(QVector<QPoint> cars);    // x=위치, y=차선
```

`QPoint` 를 쓰면 등록 없이 바로 signal에 넘길 수 있어. `QVector<QPoint>` 는 등록이 필요할 수 있으니 `qRegisterMetaType<QVector<QPoint>>("QVector<QPoint>");` 는 그대로 두고.

qnode.cpp

```
cpp
vehicle_sub_ = node->create_subscription<road_friend::msg::CarArray>(
    "vehicle_pos", 10,
    [this](const road_friend::msg::CarArray::SharedPtr msg)
    {
        QVector<QPoint> cars;
        for (size_t i = 0; i < msg->cars.size(); i++)
        {
            cars.push_back(QPoint((int)msg->cars[i].x, msg->cars[i].lane));
        }
        Q_EMIT vehiclesUpdated(cars);
    });
```

`QPoint` 의 `x()` 에 화면 x좌표, `y()` 에 차선 번호를 담는 거야. 좀 편법인데 구조체 새로 만들 것 없이 깔끔해.

2. MainWindow에서 y 계산

cpp

차선 번호를 화면의 세로 좌표로 바꾸는 것이 핵심이라는 답을 받았다. 위치와 차선 번호를 한 쌍으로 묶어서 화면 쪽에 넘기고, 화면에서 차선 번호를 실제 좌표로 바꾸는 방식을 안 내받았다.

구조체를 새로 만들지 않고 좌표를 담는 기존 타입의 두 값을 각각 위치와 차선 번호로 쓰는 방법이 제시되었다. 편법이지만 타입을 새로 만들 필요가 없어서 그대로 따랐다.

앞차 간격이 달라진 부분 어떻게 해? 설명해줘

전에는 왜 **i-1** 로 했나

차선이 하나였을 때 배열은 이랬어:

```
vehicles_ = [280, 240, 200]
```

↑앞 ↑중간 ↑뒤

push_back 으로 뒤에 붙으니 먼저 생긴 차가 항상 앞 인덱스야. 그래서 **i** 번째 차의 앞차는 무조건 **i-1** 이 있어.

지금 왜 안 되나

차선이 섞이면서 이렇게 돼:

```
vehicles_ = [{lane:0, x:280}, {lane:2, x:240}, {lane:0, x:200}]
```

인덱스 0 인덱스 1 인덱스 2

인덱스 2번 차(lane 0)의 앞차는 인덱스 0번이야. **i-1** 인 1번은 **다른 차선**이라 상관없는 차고. 그대로 두면 3차선 차 때문에 1차선 차가 급정거하는 이상한 일이 생겨.

front 배열이 하는 일

차선 개수만큼 칸을 만들어서, 각 차선의 "가장 최근에 처리한 차 위치"를 저장해.

```
cpp
std::vector<float> front(lane_count_, 999999.0);
```

vector(개수, 초기값) 이라 **[999999, 999999, 999999]** 로 시작해.

반복문이 돌면서 이렇게 변해:

```
i=0 (lane 0, x=280) → front[0]이 999999 → 제한 없음 → front = [280, 999999, 999999]
i=1 (lane 2, x=240) → front[2]가 999999 → 제한 없음 → front = [280, 999999, 240]
```

차선이 하나였을 때는 배열에서 바로 앞 인덱스가 곧 앞차였는데, 차선이 섞이면서 그 방식이 통하지 않게 된 것이 원인이라는 답을 받았다. 바로 앞 인덱스가 다른 차선의 차일 수 있어서 엉뚱한 차 때문에 급정거하게 된다는 설명이다. 차선 개수만큼 칸을 가진 배열을 따로 두고 각 차선의 직전 차 위치를 기록해두는 방법을 안내받았다. 초기값을 아주 큰 수로 두어야 그 차선의 첫 차가 제한에 걸리지 않는다는 점도 들었는데, 처음에 0으로 두었더니 첫 차가 화면 밖으로 밀려나서 이유를 알 수 있었다.

```
[road_friend-3] invalid allocator, at ./src/rcf/subscription.c:261
[road_friend-3] terminate called after throwing an instance of
'rcpp::exceptions::RCLError'
[road_friend-3] what(): could not create subscription: invalid allocator, at
./src/rcf/subscription.c:261
[ERROR] [vehicle_node-2]: process has died [pid 43373, exit code -6, cmd
/home/raj/Documents/21th_robot_intern_LeeJuwon/ROS2/install/road_friend/lib/roa
d_friend/vehicle_node --ros-args -r _node:=vehicle_node --params-file
/home/raj/Documents/21th_robot_intern_LeeJuwon/ROS2/install/road_friend/share/
road_friend/config/vehicle.yaml]
더 보기
```

Debugging an allocator error in ROS 2 message setup.

set(CMAKE_CXX_STANDARD 11) 이 원인이야. 커스텀 메시지를 안 쓰는 traffic_light_node만 살아남은 게 단서고.

CMakeLists.txt:

```
cmake
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

```
bash
cd ~/Documents/21th_robot_intern_LeeJuwon/ROS2
rm -rf build/road_friend install/road_friend
cbb
```

메시지 생성 코드는 ROS가 기본 표준(17)으로 컴파일하는데, 네 코드에는 11로 컴파일되니까 돌아 보는 **std::vector / std::allocator** 의 내부 구조가 달라. 그래서 메시지 타임을 넘기는 순간 "invalid allocator" 가 뜨는 거야. 같은 구조체를 서로 다르게 해석하는 거라 ABI 불일치라고 불러.

std_msgs 는 ROS가 미리 빌드해둔 거라 이 문제를 안 겪었고, 그래서 traffic_sim 때는 11이여도 말썹했던 거야. Jazzy 자체가 C++17 기준이라 원래 17로 맞추는 게 맞아.

빌드 설정의 C++ 표준이 낮게 잡혀 있는 것이 원인이라는 답을 받았다. 메시지 생성 코드는 기본 표준으로 컴파일되는데 내 코드는 더 낮은 표준으로 컴파일되어서, 둘이 보는 내부 구조가 달라 메시지를 넘기는 순간 오류가 난다는 설명이다. 커스텀 메시지를 쓰지 않는 노드만 살아남은 것이 단서였다고 한다. 표준을 맞추고 빌드 폴더를 지운 뒤 다시 빌드하니 해결되었다.

include/road_friend/player_node.hpp

```
cpp

#ifndef PLAYER_NODE_HPP
#define PLAYER_NODE_HPP

#include "rclcpp/rclcpp.hpp"
#include "road_friend/msg/move.hpp"
#include "road_friend/msg/car_array.hpp"
#include "road_friend/msg/player_state.hpp"
#include "road_friend/srv/reset.hpp"

class PlayerNode : public rclcpp::Node
{
public:
    PlayerNode();

private:
    void move_callback(const road_friend::msg::Move::SharedPtr msg);
    void car_callback(const road_friend::msg::CarArray::SharedPtr msg);
    void request_reset();
};
```

이동 명령을 받는 구독자, 차량 위치를 받는 구독자, 자기 상태를 알리는 발행자, 리셋을 요청하는 클라이언트를 갖는 구조를 안내받았다. 위치와 생존 여부, 점수를 멤버 변수로 들고 있으면서 각 콜백에서 갱신하는 방식이다. 이동과 충돌을 각각 다른 콜백에서 처리하게 되어 있어서, 하나의 기능을 통째로 짜는 것보다 나누어 생각하기 편했다.

player가 순간이동하는데 왜 그래?

상태가 바뀌긴 했는데 화면에 남아 있는 시간이 너무 짧아서 그렇게 보일 수 있어.
충돌 → 리셋 흐름이 이런 순서라면.

```
cpp
alive_ = false;
publish_state();    // 죽음 알림
request_reset();    // 바로 리셋 요청
```

서비스 응답이 1000브의 몇 초 만에 돌아와, 그 사이에 화면은 한 번도 안 그려지거나, 그려지도 눈에 안 띄는 순간에 지나가, 결과만 보면 차에 닿는 순간 캐릭터가 시작 위치로 튀는 것처럼 보여.

확인 방법 — 노드 터미널에 죽었다는 로그가 찍히는지 보. 찍히는데 화면에서 안 보였으면 이 경우야.

해결

리셋을 일주일 늦추면 돼. 타이머를 하나 걸어서 1초 뒤에 요청하는 거야.

```
cpp
//.hpp
rclcpp::TimerBase::SharedPtr reset_timer_;
```

```
cpp
alive_ = false;
publish_state();
```

```
reset_timer_ = this->create_wall_timer(
    std::chrono::seconds(1),
    [this]()
    {
        reset_timer_->cancel();
        request_reset();
    });
```

cancel() 을 먼저 부르는 게 중요해. 타이머는 원래 주기적으로 계속 도는 물건이라, 안 끄면 1초마다 리셋 요청이 계속 나.

이러면 회색으로 변한 캐릭터가 1초 동안 그 자리에 머물다가 시작 위치로 돌아가. 플레이어 입장에서 "여기서 죽었는지" 보니까 게임으로서 자연스러워지고.

다른 가능성도 있어. 화면에 그리는 대상이 여러 개인데 목록에서 중간 것이 빠지면, 뒤에 있던 것이 앞으로 당겨지면서 같은 자리에 다른 대상이 그려져, 그것도 순간이동처럼 보이거든.

상태가 바뀌긴 했는데 화면에 남아 있는 시간이 너무 짧아서 그렇게 보이는 것이라는 답을 받았다. 서비스 응답이 아주 빨리 돌아와서 죽은 상태가 화면에 그려지기 전에 지나간다는 설명이다.

노드 터미널에 죽었다는 로그가 찍히는지 확인해보라는 안내를 받았고, 실제로 찍히고 있어서 원인을 확정할 수 있었다. 타이머를 걸어 1초 뒤에 리셋을 요청하도록 바꾸니 죽은 위치가 보이게 되었다. 타이머는 원래 계속 도는 것이라 한 번만 실행되게 꺼줘야 한다는 점도 함께 들었다.

타입서포트 라이브러리가 없다는데 왜

커스텀 메시지를 만들면 파일이 한 종류만 생기는 게 아니야.

```
libroad_friend__rosidl_typesupport_cpp.so      ← C++ 코드용
libroad_friend__rosidl_typesupport_fastrtps_cpp.so ← 실제 통신용
libroad_friend__rosidl_typesupport_introspection_cpp.so
```

첫 번째는 코드에서 `msg.x = 10` 같은 걸 쓸 수 있게 해주는 거고, 두 번째는 그 메시지를 실제로 네트워크에 실어 보낼 때 쓰는 거야. ROS2는 DDS라는 통신 미들웨어 위에서 도는데, 메시지를 바이트로 바꾸는 방법을 DDS가 알아야 하거든.

컴파일은 첫 번째만 있어도 통과해. 그래서 빌드는 성공하는데 실행하면 죽는 상황이 나와. 발행자나 구독자를 만드는 순간 DDS용 라이브러리를 찾다가 없으니까 터지는 거야.

확인

```
bash
ls install/패키지명/lib/ | grep typesupport
```

`fastrtps_cpp` 가 없으면 이 경우야.

원인 1: package.xml

```
xml
<member_of_group>roscpp_interface_packages</member_of_group>
```

이게 빠지면 빌드 시스템이 "이 패키지는 인터페이스를 제공한다"는 걸 몰라서 DDS용 생성을 건너뛰어. 같이 필요한 것들:

```
xml
<buildtool_depend>roscpp_default_generators</buildtool_depend>
<exec_depend>roscpp_default_runtime</exec_depend>
```

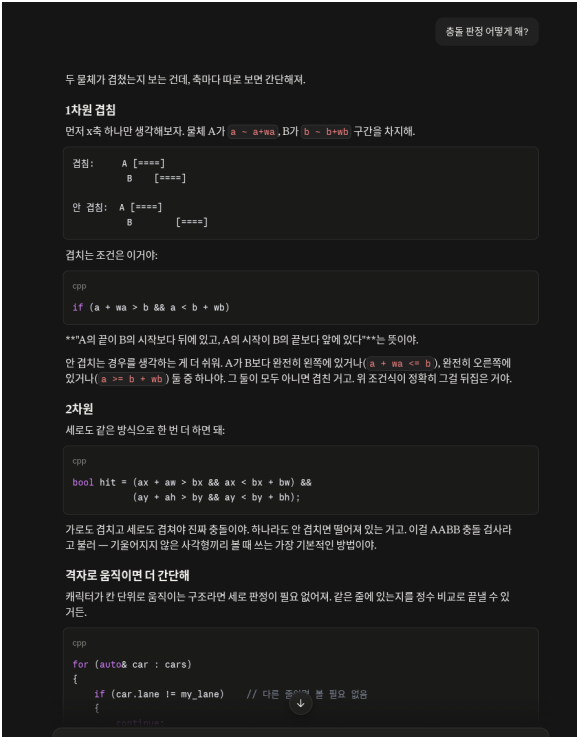
원인 2: 타겟 이름

```
cmake
roscpp_generate_interfaces(${PROJECT_NAME}_msgs # ← 이러면 안 됨
    "msg/Car.msg"
)
```

커스텀 메시지를 만들면 라이브러리가 한 종류만 생기는 것이 아니라는 답을 받았다. 코드에서 쓰는 것과 실제 통신에 쓰는 것이 따로 있는데, 컴파일은 앞의 것만 있어도 통과하기 때문에 빌드는 성공하고 실행만 실패하는 상황이 나온다는 설명이다.

원인으로 두 가지를 안내받았다. 하나는 패키지 설정에 인터페이스 패키지라는 표시가 빠진 경우, 다른 하나는 메시지를 생성하는 부분의 이름을 패키지 이름과 다르게 준 경우다.

내 경우는 두 번째였다. 실행 파일 이름이 이미 패키지 이름과 같아서 충돌을 피하려고 메시지 쪽 이름에 접미사를 붙였던 것이 문제였다. 메시지 쪽은 이름을 바꿀 수 없으니 실행 파일 이름을 바꾸라는 안내를 받아 그렇게 고쳤다.



두 물체가 겹쳤는지는 축마다 따로 보면 간단해진다는 답을 받았다. 각 물체가 차지하는 구간을 놓고, 한쪽의 끝이 다른 쪽의 시작보다 뒤에 있으면서 시작이 상대의 끝보다 앞에 있으면 겹친 것이다. 안 겹치는 경우를 먼저 생각하면 이해하기 쉽다는 설명도 들었다. 완전히 왼쪽에 있거나 완전히 오른쪽에 있는 두 경우가 아니면 겹친 것이다. 캐릭터가 칸 단위로 움직이는 구조라면 같은 줄인지를 정수 비교로 먼저 거를 수 있어서 가로 판정 하나만 남는다는 점도 안내받았고, 내 경우가 그래서 그대로 구현했다.



점수를 아는 노드와 차를 움직이는 노드가 달라서 그 사이에 연결을 만들어야 한다는 답을 받았다. 차량 노드가 플레이어 상태를 구독하면 되고, 이미 화면에 그리려고 발행하고 있는 토픽이라 새로 만들 필요는 없다고 한다. 토픽은 구독자가 몇 명이든 상관없기 때문이다. 속도를 직접 덮어쓰지 않고 배율을 따로 두는 이유도 들었다. 차선마다 속도가 다른 경우 원본을 덮어쓰면 되돌릴 수 없어서, 배율을 곱하는 방식이라야 원본을 유지하면서 전체를 비례해 올릴 수 있다.

3. 깨달은 점

빌드가 성공해도 실행이 안 되는 경우가 있다는 것을 알게 되었다. 이번에 두 번 겪었는데 표준이 안 맞았던 것과 통신용 라이브러리가 없었던 것이 그랬다. 둘 다 컴파일 단계에서는 걸리지 않고 발행자나 구독자를 만드는 순간에 터졌다. 오류 메시지만 보면 원인을 짐작하기 어려웠는데, 어떤 노드는 살아남고 어떤 노드는 죽었는지를 보면 범위를 좁힐 수 있다는 것을 배웠다.

일반적인 규칙을 그대로 쓸 수 없는 경우가 있다는 것을 알게 되었다. 메시지 생성 부분의 이름은 패키지 이름과 같아야 하는데 내 패키지는 그 이름의 실행 파일이 이미 있었다. 규칙을 비틀어서 접미사를 붙였더니 실행 시점에 문제가 생겼고, 결국 양보할 수 없는 쪽은 그대로 두고 실행 파일 이름을 바꾸는 쪽으로 해결했다. 어느 쪽이 바꿀 수 있는 것인지 따져봐야 한다는 점을 느꼈다.

콜백 안에서 무언가를 기다리면 안 된다는 제약이 있다는 것을 알게 되었다. 노드가 하나의 루프로 돌아간다는 구조를 알고 나니 왜 멈추는지 납득이 되었다. 기다리는 대신 나중에 실행할 함수를 넘기는 방식이 있다는 것도 이번에 처음 써봤다.

문제의 원인이 코드가 아니라 타이밍일 수 있다는 것을 알게 되었다. 플레이어가 순간이동하는 것처럼 보였을 때 처음에는 위치 계산이 틀렸다고 생각했는데, 로그를 확인해보니 상태는 정상적으로 바뀌고 있었고 너무 빨라서 안 보였던 것뿐이었다. 화면에 안 보인다고 동작이 안 하는 것은 아니라는 점을 알았다.

노드 사이의 데이터가 한 방향으로만 흐르지 않아도 된다는 것을 알게 되었다. 처음에는 차량 노드가 플레이어에게 정보를 주기만 하는 줄 알았는데, 난이도를 올리려고 하니 반대 방향 구독이 필요했다. 이미 발행하고 있는 토픽에 구독자를 하나 더 붙이는 것만으로 되어서 생각보다 간단했다.

어떤 문제는 차원을 나누면 쉬워진다는 것을 알게 되었다. 충돌 판정이 그랬는데, 가로와 세로를 한꺼번에 생각하면 복잡해 보이지만 축마다 따로 보면 각각은 단순한 비교였다. 내 경우에는 칸 단위로 움직여서 세로는 정수 비교로 끝나 더 간단해졌다.